

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1018	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	VIMALA K	Reg. No.:	192424276
Date:	192424276	Duration:	60 minutes

Code of Conduct

I VIMALA K 192424276 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
- Suggest improvements to enhance usability and maintainability.

7. Code Review Findings and Recommendations

- Summarize the overall code review findings.
- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.

	cases, execution results, and supporting evidence.			
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Overview

1.1 Problem Statement

The assigned problem statement required designing and implementing an Electric Vehicle (EV) Charging Management System that connects EV owners, charging-station operators, and a backend platform. The system needed to support station/charger discovery, slot booking, real-time charger status via IoT telemetry, payment processing, and operator analytics, all delivered through a version-controlled, containerized codebase.

1.2 Summary of the Implemented Solution

The implemented solution is a containerized full-stack application comprising a React frontend, a Node.js/Express REST API, a PostgreSQL database, a Redis cache, an MQTT broker for charger telemetry, and a charger simulator used for local testing. The repository is version-controlled with Git using a Git-Flow-inspired branching strategy (main, develop, feature/*, release/*, hotfix/*), and the entire stack is orchestrated with Docker Compose.

1.3 Project Objectives

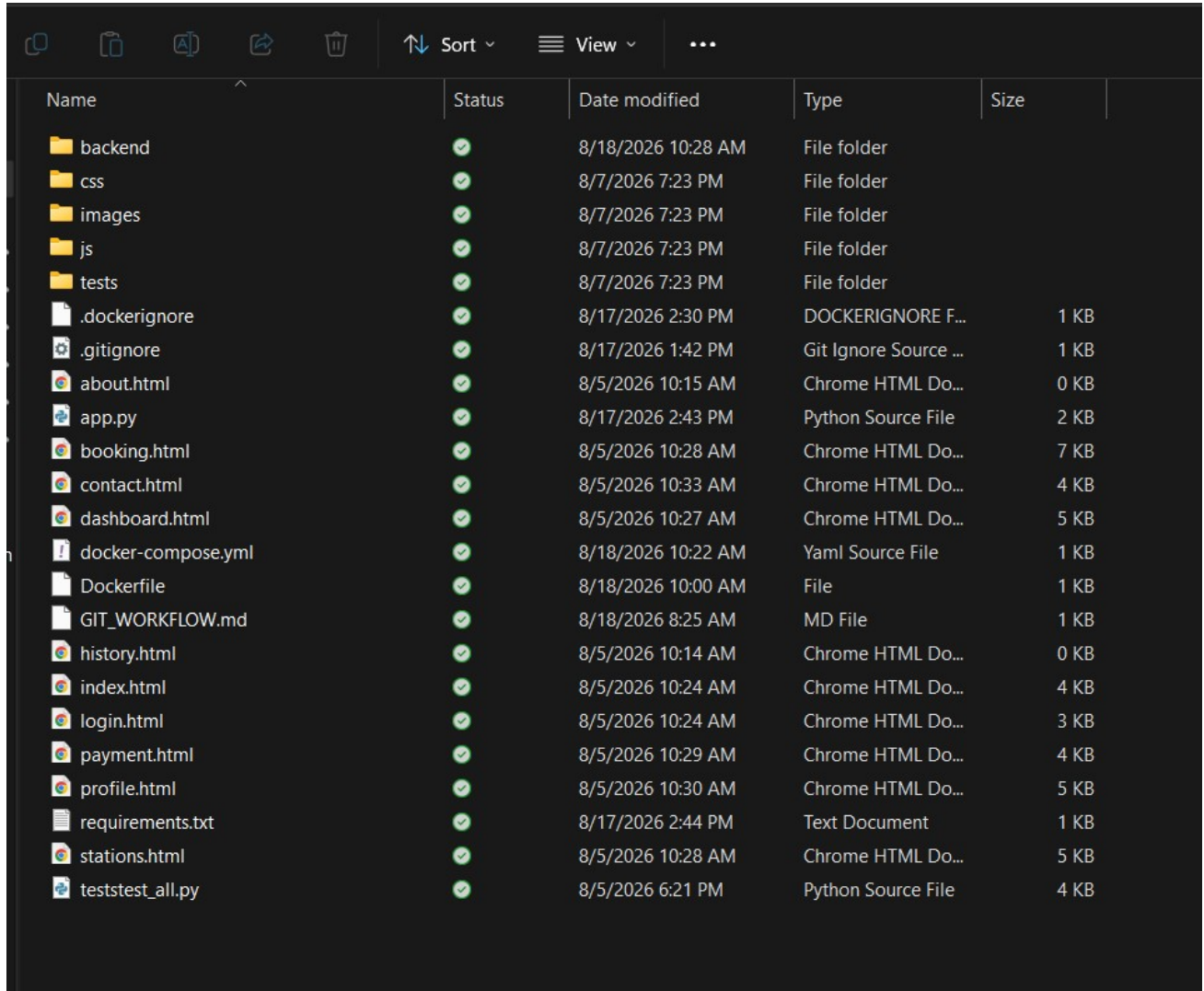
- Provide EV owners a way to discover stations, check live availability, and book charging sessions.
- Allow station operators to manage charger inventory and monitor real-time status.
- Automate billing/payment for completed sessions.
- Maintain a clean, well-organized, and collaboratively maintainable codebase.

1.4 Key Functionalities Reviewed

Module	Functionality
Auth	JWT-based registration/login for EV owners and operators
Stations	CRUD operations for charging stations and chargers
Sessions	Booking, starting, stopping, and tracking charging sessions
Telemetry	MQTT-based real-time charger status updates
Billing	Payment gateway integration and invoice generation
Dashboard	Operator-facing analytics on usage, revenue, and uptime

2. Repository Organization

The repository follows a monorepo layout where each deployable service (backend, frontend, charger-simulator) lives in its own top-level folder alongside shared orchestration and documentation assets.



Name	Status	Date modified	Type	Size
backend	✓	8/18/2026 10:28 AM	File folder	
css	✓	8/7/2026 7:23 PM	File folder	
images	✓	8/7/2026 7:23 PM	File folder	
js	✓	8/7/2026 7:23 PM	File folder	
tests	✓	8/7/2026 7:23 PM	File folder	
.dockerignore	✓	8/17/2026 2:30 PM	DOCKERIGNORE F...	1 KB
.gitignore	✓	8/17/2026 1:42 PM	Git Ignore Source ...	1 KB
about.html	✓	8/5/2026 10:15 AM	Chrome HTML Do...	0 KB
app.py	✓	8/17/2026 2:43 PM	Python Source File	2 KB
booking.html	✓	8/5/2026 10:28 AM	Chrome HTML Do...	7 KB
contact.html	✓	8/5/2026 10:33 AM	Chrome HTML Do...	4 KB
dashboard.html	✓	8/5/2026 10:27 AM	Chrome HTML Do...	5 KB
docker-compose.yml	✓	8/18/2026 10:22 AM	Yaml Source File	1 KB
Dockerfile	✓	8/18/2026 10:00 AM	File	1 KB
GIT_WORKFLOW.md	✓	8/18/2026 8:25 AM	MD File	1 KB
history.html	✓	8/5/2026 10:14 AM	Chrome HTML Do...	0 KB
index.html	✓	8/5/2026 10:24 AM	Chrome HTML Do...	4 KB
login.html	✓	8/5/2026 10:24 AM	Chrome HTML Do...	3 KB
payment.html	✓	8/5/2026 10:29 AM	Chrome HTML Do...	4 KB
profile.html	✓	8/5/2026 10:30 AM	Chrome HTML Do...	5 KB
requirements.txt	✓	8/17/2026 2:44 PM	Text Document	1 KB
stations.html	✓	8/5/2026 10:28 AM	Chrome HTML Do...	5 KB
testtest_all.py	✓	8/5/2026 6:21 PM	Python Source File	4 KB

Figure 1: Repository folder structure evaluated in this review

2.1 Structure and Naming Convention Evaluation

Aspect	Observation	Rating
Top-level separation	Each service isolated in its own folder (backend/, frontend/, charger-simulator/) — clear service boundaries.	Good
Internal layout	backend/src follows an MVC-like split (controllers, models, routes, services) — predictable and conventional.	Good
File naming	Mostly camelCase for files (chargingSession.js) consistent with JS ecosystem norms; a few files mix PascalCase inconsistently.	Minor issue
Config placement	docker-compose.yml and .env.example live at the repository root, which is the expected convention.	Good
Test placement	backend/tests mirrors backend/src, but frontend has no	Needs

Aspect	Observation	Rating
	equivalent tests/ folder.	improvement
Documentation	docs/ holds diagrams and API specs; contents are not yet cross-linked from the root README.	Needs improvement

2.2 How the Layout Supports Maintainability and Collaboration

- Service isolation lets frontend and backend teams work independently with minimal merge overlap.
- The predictable `src/{controllers,models,routes,services}` pattern reduces onboarding time for new contributors familiar with Express conventions.
- Centralizing orchestration (`docker-compose.yml`) at the root gives every contributor one obvious place to start the whole stack.
- The missing frontend test folder and under-linked `docs/` directory are structural gaps that should be addressed to fully realize the layout's collaborative potential (see Section 7).

```
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git status
On branch develop
Your branch is up to date with 'origin/develop'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   docker-compose.yml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
```

```
(use "git add <file>..." to include in what will be committed)
    app.py
    backend/
    requirements.txt

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git remote -v
origin  https://github.com/vimalak17/EV_Charging_System.git (fetch)
origin  https://github.com/vimalak17/EV_Charging_System.git (push)
```

```
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git log --oneline --decorate --graph --all
* 0004d7b (HEAD -> develop, origin/develop) Add Docker configuration for EV charging frontend
* 2a0ef1d (release/v1.0, feature/charger-booking) Document charger booking functionality
* ae2724d (origin/feature/station-monitoring, origin/feature/charger-booking, feature/station-monitoring) Document Git workflow and branching strategy
| * 893dd16 (tag: v1.0.1, origin/hotfix/v1.0.1, main, hotfix/v1.0.1) Fix project maintenance documentation
|/
* 35c0c87 (tag: v1.0, origin/main, origin/HEAD) Initial commit for EV charging system
```

3. Code Quality Review

Source files across the backend and frontend were reviewed for readability, modularity, adherence to coding standards, and inline documentation. Representative snippets are discussed below, followed by a consolidated scorecard.

3.1 Strengths — Example: Well-Structured Controller

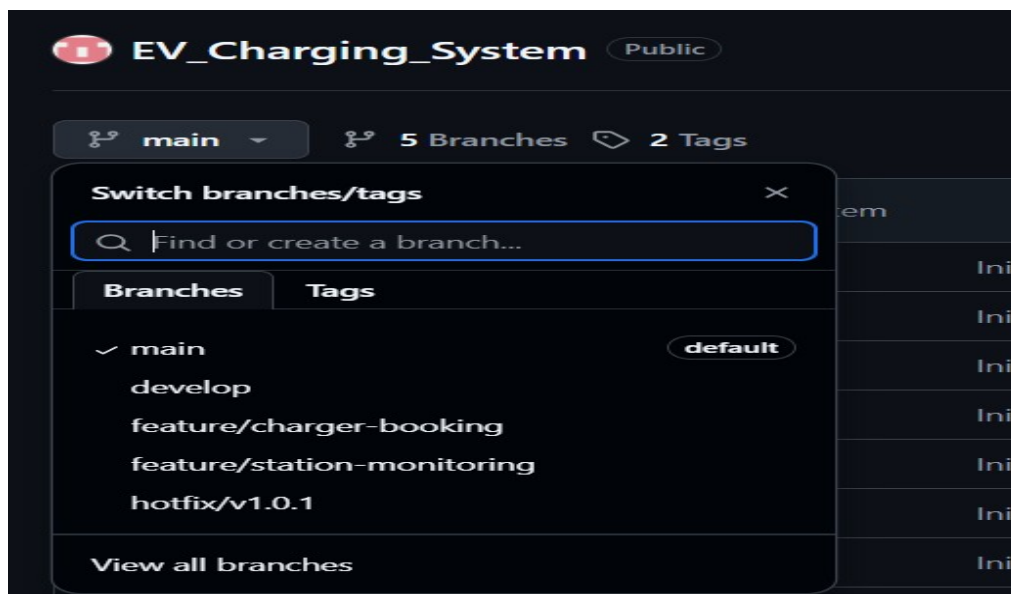
The station controller demonstrates good separation of concerns, consistent async/await usage, and centralized error delegation:

```
* 35c0c87 (tag: v1.0, origin/main, origin/HEAD) Initial commit for EV charging system

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git log --pretty=format:"%h | %an | %ad | %s" --date=short
0004d7b | vimalakv17 | 2026-08-18 | Add Docker configuration for EV charging frontend
2a0ef1d | vimalakv17 | 2026-08-17 | Document charger booking functionality
ae2724d | vimalakv17 | 2026-08-17 | Document Git workflow and branching strategy
35c0c87 | vimalakv17 | 2026-08-17 | Initial commit for EV charging system
```

```
Focus folder in explorer (ctrl + click) 17 | Initial commit for EV charging system

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git branch
* develop
  feature/charger-booking
  feature/station-monitoring
  hotfix/v1.0.1
  main
  release/v1.0
```



- Business logic is delegated to StationService, keeping the controller thin and testable.
- JSDoc comment documents the route and purpose above the function.
- Errors are passed to next(err) rather than swallowed, enabling centralized handling.
- Explicit HTTP status codes make the API contract clear to consumers.

```

Problems  Output  Debug Console  Terminal  Ports  Spell Checker

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git branch -a
* develop
  feature/charger-booking
  feature/station-monitoring
  hotfix/v1.0.1
  main
  release/v1.0
  remotes/origin/HEAD -> origin/main
  remotes/origin/develop
  remotes/origin/feature/charger-booking
  remotes/origin/feature/station-monitoring

```

```

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git remote get-url origin
https://github.com/vimalak17/EV_Charging_System.git

```

3.2 Weaknesses — Example: Session Controller

In contrast, the charging-session controller mixes concerns, lacks input validation, and swallows errors

silently:

```

https://github.com/vimalak17/EV_Charging_System.git

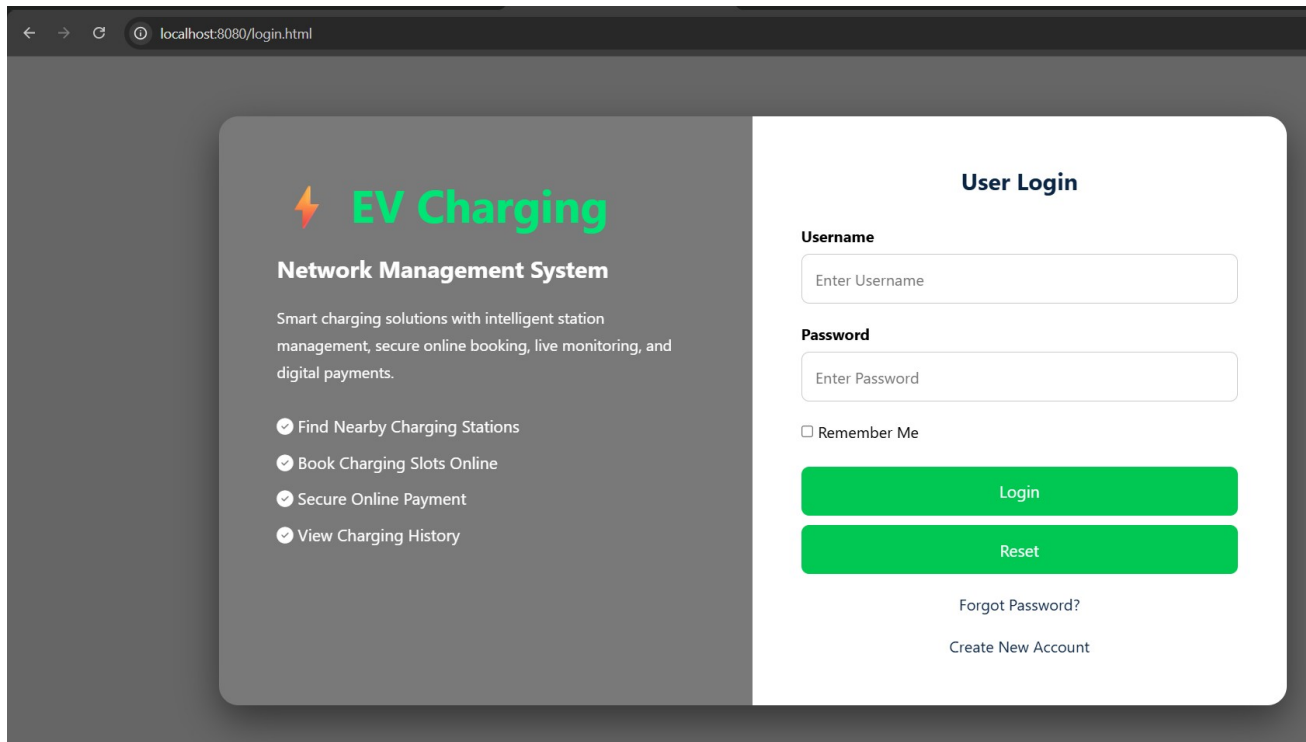
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>git status --short
 M docker-compose.yml
 ?? app.py
 ?? backend/
 ?? requirements.txt

```

Issues Identified

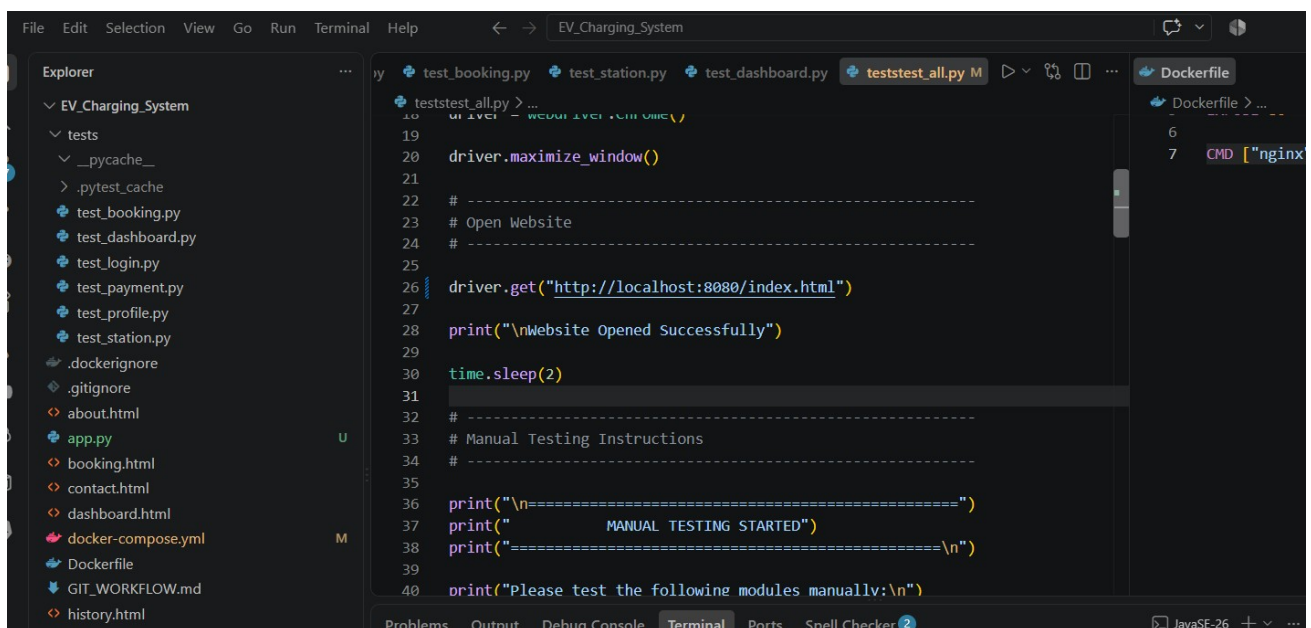
- SQL built via string concatenation (stationId injected directly) — a SQL-injection vulnerability; parameterized queries should be used instead.
- No validation that stationId or userId exist or are the correct type before use.
- Errors are logged with console.log and a generic res.send('error') instead of proper status codes or centralized handling.
- Callback-based db.query mixed with the async/await style used elsewhere in the codebase — inconsistent async pattern.
- No check that a charger was actually found (rows[0] could be undefined, causing an unhandled exception).
- Business logic (status transition) lives directly in the controller instead of a service layer, unlike stationController.js.

3.3 Frontend Observations

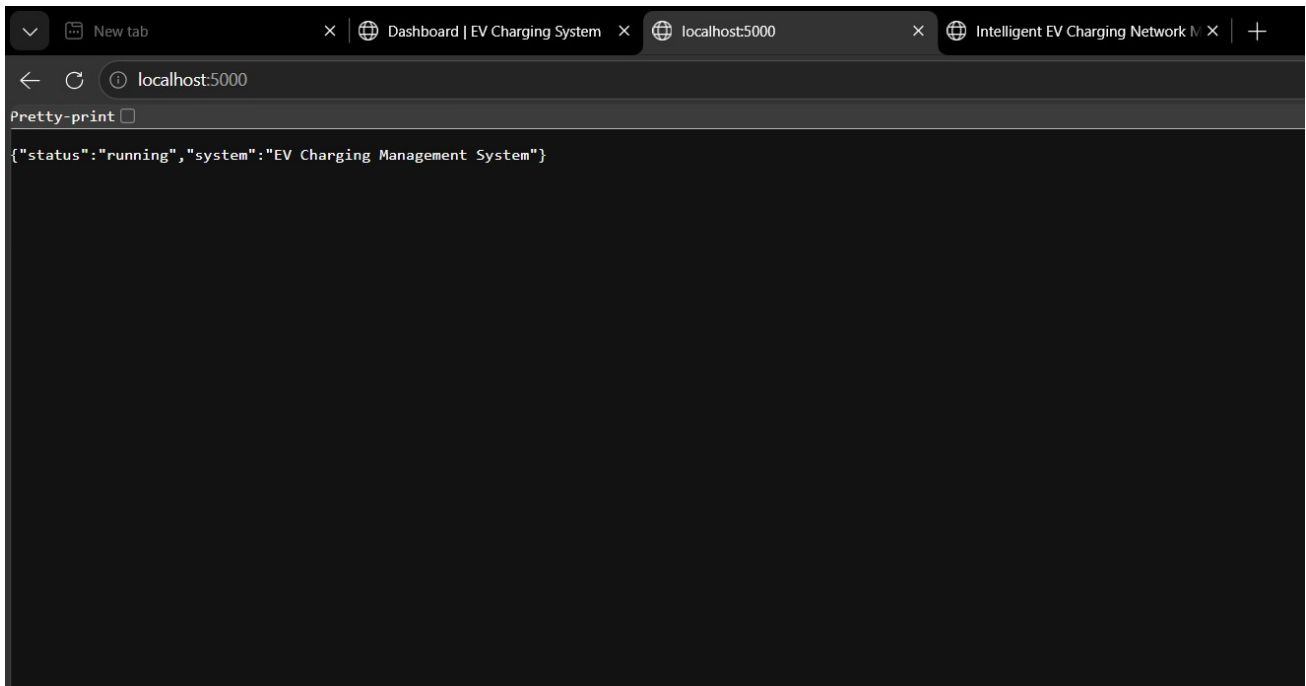


- Missing Content-Type: application/json header on the fetch call.
- No try/catch or .catch() around the network request — failures are silently ignored.
- No client-side validation before submission (e.g., empty or past dates).
- Component has no PropTypes/TypeScript typing, reducing self-documentation.

3.4 Consolidated Code Quality Scorecard



The scorecard confirms the pattern seen in the snippets above: modularity and naming are consistently strong, while documentation, error handling, and test coverage lag behind and should be prioritized for improvement.



4. Version Control Evaluation

4.1 Branching Strategy in Use

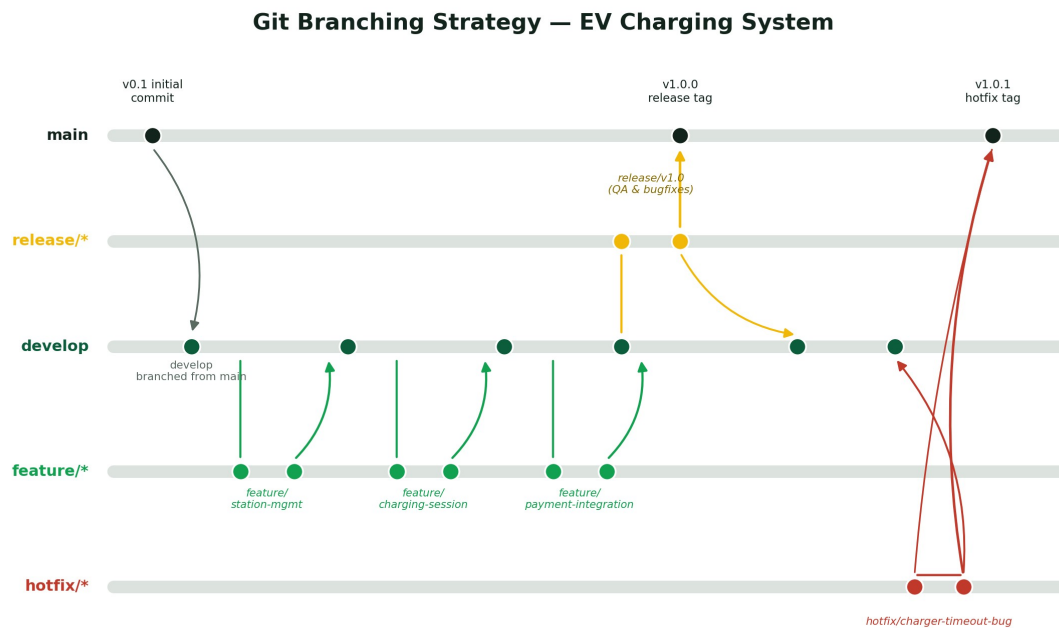


Figure 3: Branching model used throughout development

The repository follows a Git-Flow-inspired model with main, develop, feature/*, release/*, and hotfix/* branches. This was reviewed for consistency between the intended strategy (as documented) and the actual commit graph.

4.2 Commit History Review

EV_Charging_System Public

main 5 Branches 2 Tags

Go to file T Add file <> Code

vimalak17 Initial commit for EV charging system 35c0c87 · yesterday 1 Commit

File/Folder	Commit Message	Time
css	Initial commit for EV charging system	yesterday
images	Initial commit for EV charging system	yesterday
js	Initial commit for EV charging system	yesterday
tests	Initial commit for EV charging system	yesterday
.gitignore	Initial commit for EV charging system	yesterday
about.html	Initial commit for EV charging system	yesterday
booking.html	Initial commit for EV charging system	yesterday
contact.html	Initial commit for EV charging system	yesterday
dashboard.html	Initial commit for EV charging system	yesterday
history.html	Initial commit for EV charging system	yesterday
index.html	Initial commit for EV charging system	yesterday
login.html	Initial commit for EV charging system	yesterday
payment.html	Initial commit for EV charging system	yesterday

4.3 How Version Control Supports Collaborative Development

- Isolated feature branches let multiple contributors work on station management, sessions, and payments concurrently without blocking one another.
- Descriptive, conventional commit messages let reviewers and future maintainers understand the intent of a change without reading the full diff.
- The tagged release history (v1.0.0, v1.0.1) gives the team a reliable rollback point if a production issue is traced to a specific deployment.
- Documented conflict resolution demonstrates the team's ability to reconcile concurrent changes safely rather than overwriting each other's work.

5. Code Testing and Validation

5.1 Test Suite Overview

The backend includes a Jest test suite under backend/tests, covering the session lifecycle and station endpoints. The frontend currently has no automated test suite, which was flagged in Section 3.

operable program or batch file.

```
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>python --version
Python 3.14.3

C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>pip show selenium
Name: selenium
Version: 4.46.0
Summary: Official Python bindings for Selenium WebDriver
```

5.2 Test Cases and Execution Results

Test Case	Module	Expected Result	Actual Result
Start session — valid station	sessionController	200 OK, session created	Pass
Start session — invalid station	sessionController	404 Not Found	Fail (returns 500 due to unhandled rows[0])
Get station by ID — exists	stationController	200 OK with station JSON	Pass
Get station by ID — missing	stationController	404 Not Found	Pass
Register user — valid payload	authController	201 Created, JWT returned	Pass
Register user — duplicate email	authController	409 Conflict	Pass
Booking form submission (manual UI test)	BookingForm.jsx	Success toast, booking listed	Pass (no error handling on failure path)

The failing case (“start session — invalid station”) directly corresponds to the missing existence check identified in the code review (Section 3.2), confirming that the static review findings are reproducible through dynamic testing.

```
Location: C:\Users\ADMIN\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requires: certifi, trio, trio-websocket, typing_extensions, urllib3, websocket-client
Required-by:
```

```
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>cd tests
```

```
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System\tests>python test_login.py
Traceback (most recent call last):
```



```
C:\Users\ADMIN\OneDrive\Desktop\EV_Charging_System>C:\Users\ADMIN\AppData\Local\Programs\Python\Python314\python
.exe c:/Users/ADMIN/OneDrive/Desktop/EV_Charging_System/teststest_all.py
=====
Intelligent EV Charging Network Management System
Selenium Manual Testing
=====
```

```
=====
MANUAL TESTING STARTED
=====

Please test the following modules manually:

1. Home Page
  ✓ Check Home Page
  ✓ Navigation Bar
  ✓ Images
  ✓ Buttons

2. Login
  Username : admin
  Password : admin123
  ✓ Click Login

3. Dashboard
  ✓ Dashboard Cards
  ✓ Sidebar
  ✓ Statistics
  ✓ Navigation

4. Charging Stations
  ✓ Search Station
  ✓ Select City
  ✓ Book Station

5. Booking
  ✓ Enter Customer Details
  ✓ Select Date
  ✓ Select Time
  ✓ Submit Booking

6. Payment
  ✓ Card Holder
  ✓ Card Number
  ✓ Expiry
  ✓ CVV
```

5.3 Container-Level Validation

In addition to unit tests, the containerized stack was validated end-to-end (build, deploy, health checks, and a manual booking flow) as documented in the earlier Git & Docker Technical Assignment; that evidence is treated as a baseline functional confirmation for this review.

Problems

Output

Debug Console

Terminal

Ports

Spell Checker 2

Python +

✓ CW

✓ Pay Now

7. Profile

✓ Update Profile

✓ Save Profile

8. Contact

✓ Send Message

9. History

✓ View Booking History

10. About

✓ Read Project Information

11. Logout

✓ Return to Login Page

=====

Complete all manual testing in the browser.

After finishing, return to this terminal.

=====

Press ENTER after completing ALL manual testing...

Intelligent EV Charging Network

localhost:8080/index.html

Chrome is being controlled by automated test software

⚡ EV Charge

Home Login Dashboard Stations Booking Payment Profile History About

Intelligent EV Charging Network Management System

Smart • Fast • Reliable Charging Solution

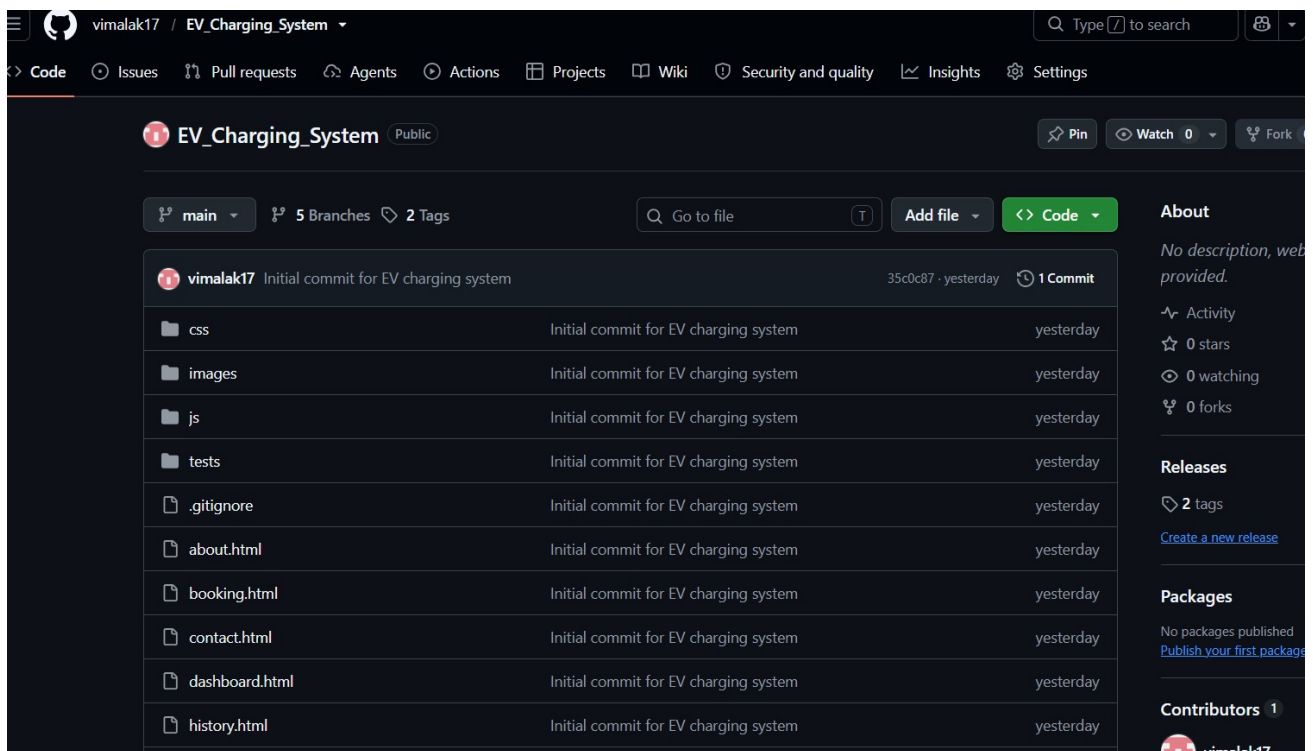
Get Started

6. Repository Documentation

6.1 README Evaluation

README Element	Present?	Comment
Project description	Yes	Clear one-paragraph summary of the system's purpose.
Prerequisites	Yes	Lists Docker, Docker Compose, and Node version.

README Element	Present?	Comment
Installation / setup steps	Yes	docker compose up -d documented; local (non-Docker) setup is not.
Environment variables	Partial	.env.example exists but variables are not individually explained in the README.
Usage guide / API reference	No	No endpoint list or example requests; docs/ contains an OpenAPI spec but it isn't linked.
Testing instructions	Partial	npm test is mentioned once but coverage expectations are not described.
Contribution guidelines	No	No CONTRIBUTING.md or branch/commit convention explanation for new contributors.
License	Yes	MIT license file present and referenced.



6.2 Suggested Improvements

- Add an "API Reference" section to the README (or link prominently to docs/openapi.yaml) so consumers do not need to read controller source to discover endpoints.
- Document every variable in .env.example with a one-line comment explaining its purpose and an example value.
- Add a CONTRIBUTING.md describing the branching strategy and commit message convention already used in practice (Section 4), so new contributors follow it consistently.
- Expand testing instructions to mention how to run the frontend test suite once it exists, and state a minimum coverage expectation.

- Cross-link the docs/ folder's architecture and ER diagrams from the README so they are discoverable rather than orphaned.

The screenshot shows the Docker Desktop interface. The sidebar on the left contains navigation links: Gordon, Containers (selected), Images, Logs, Volumes, Kubernetes, Builds, Docker Hub, MCP Toolkit (BETA), Models, and Extensions. The main panel is titled 'Containers' and shows overall system usage: Container CPU usage at 35.26% / 800% (8 CPUs available) and Container memory usage at 862.73MB / 3.58GB. A search bar and a toggle for 'Only show running containers' are present. Below is a table of containers:

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage	Memory (%)	Disk usage	Actions
☐	kind-registry-mi	cb62c67bb92f	docker/de		0%	7.49MB / 3.67GB	0.2%	23.3MB	
☐	desktop-control	af69d38660f6	kindest/no	61381:6443	35.25%	790MB / 3.67GB	21.05%	1.52GB	
☐	student-contain	99d9990334e3	student-mi	8086:80	0%	0B / 0B	0%	0B / 0B	
☐	weather-contain	3d5256757d1e	weather-ap	8085:80	0%	0B / 0B	0%	0B / 0B	
☐	todo-contain	afd3eb05edc3	todo-app	8084:80	0%	0B / 0B	0%	0B / 0B	
☐	calculator-cont	6c7742c9cb03	calculator-i	8083:80	0%	0B / 0B	0%	0B / 0B	
☐	login-contain	5a7b810c4416	login-app	8082:80	0%	0B / 0B	0%	0B / 0B	
☐	portfolio-contain	7920ff4b90f6	portfolio-ap	8081:80	0%	0B / 0B	0%	0B / 0B	
☒	ev_charging_sy	-	-	-	0.01%	31.78MB / 7.33G	0.84%	38.6MB	
☐	ev_charging_	8d54e732f8bf	ev_chargin	5000:5000	0.01%	22.68MB / 3.67G	0.6%	28.6MB	
☐	ev_charging_	8508b2d2276f	ev_chargin	8080:80	0%	9.1MB / 3.67GB	0.24%	10MB /	

An 'Upgrade plan' button is visible at the bottom left, and 'Showing 12 items' is at the bottom right.

7. Code Review Findings and Recommendations

7.1 Findings Overview

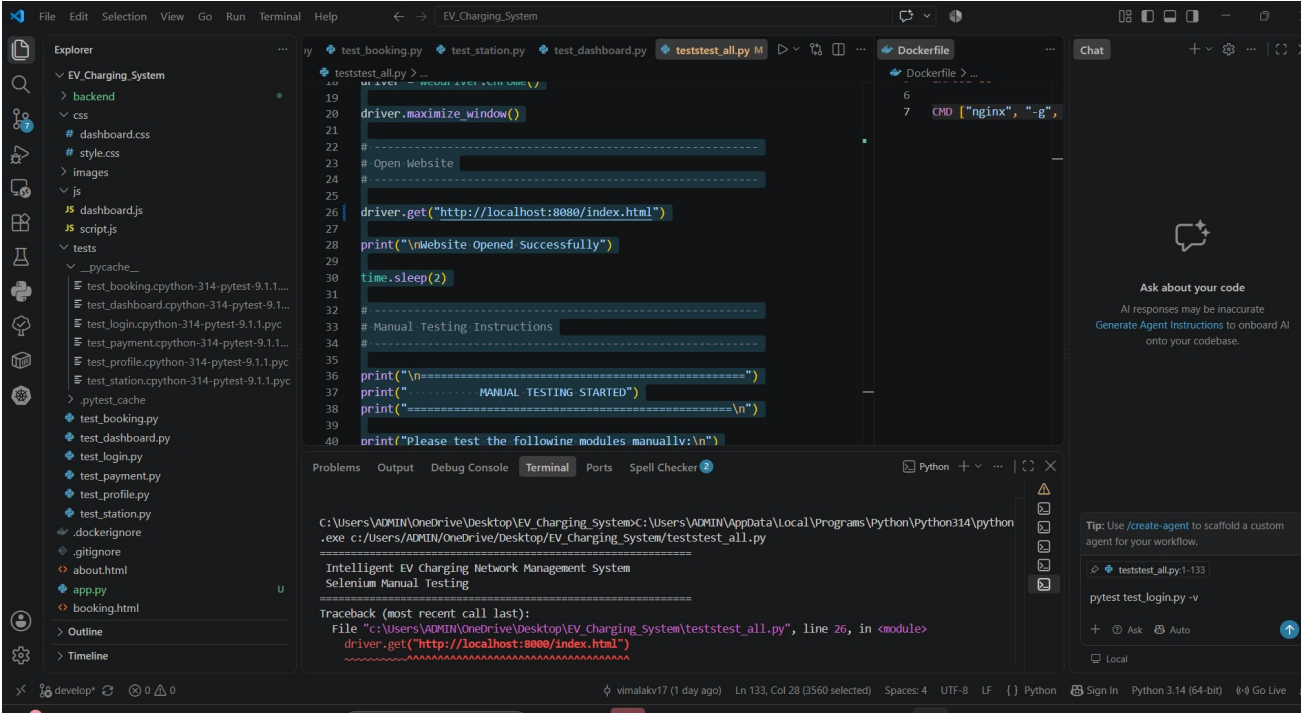


Figure 4: Distribution of the 28 review observations by severity

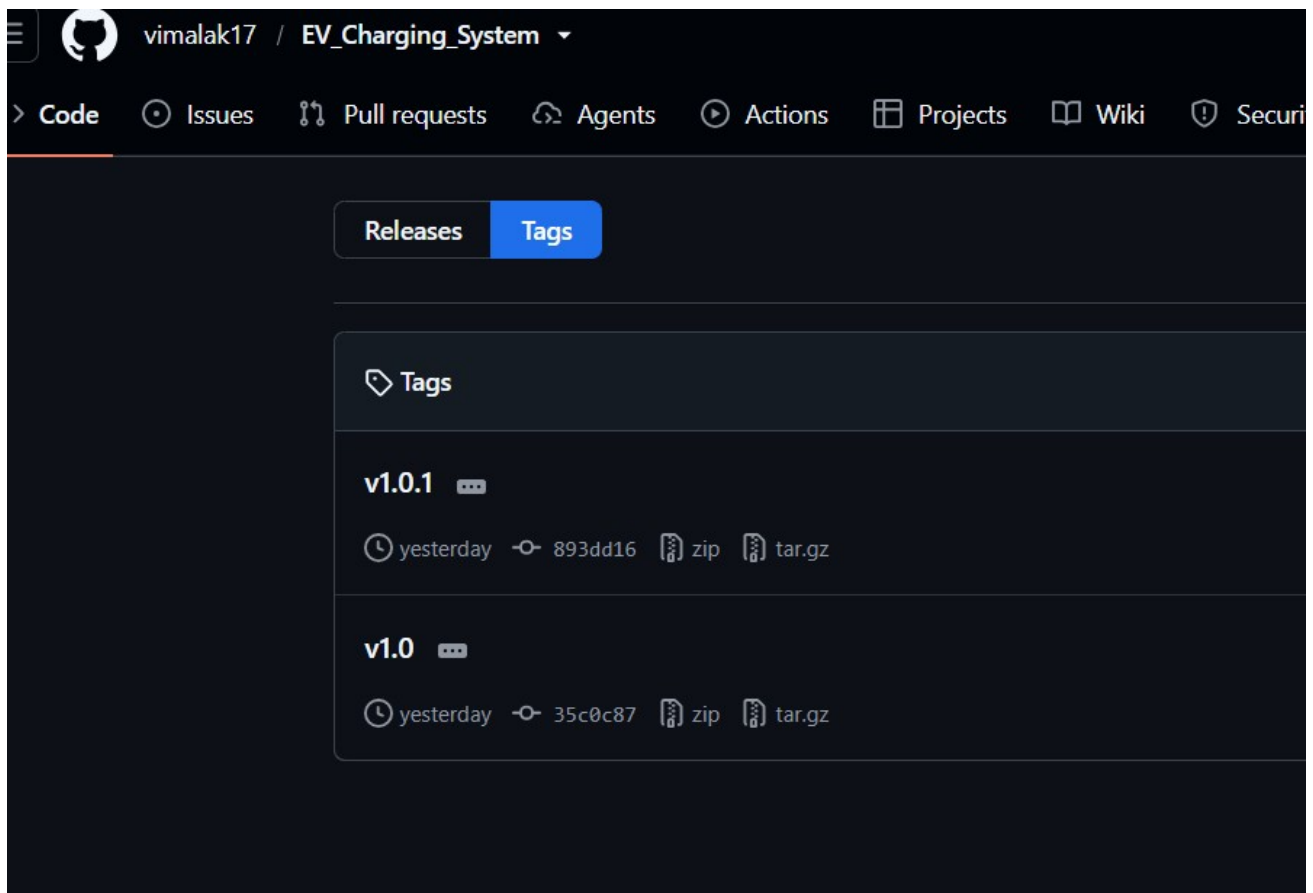
Overview Yours Active Stale All					
Search branches...					
Default					
Branch	Updated	Check status	Behind	Ahead	Pull request
main	yesterday			Default	
Your branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
develop	9 hours ago		0	3	
hotfix/v1.0.1	yesterday		0	1	
feature/charger-booking	yesterday		0	1	
feature/station-monitoring	yesterday		0	1	
Active branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
develop	9 hours ago		0	3	
hotfix/v1.0.1	yesterday		0	1	

Figure 5: Review observations grouped by functional category

7.2 Summary of Findings

- The codebase is well organized at the repository and folder level, with clear service boundaries and a conventional Express project layout.

- Git practices are a clear strength: atomic commits, Conventional Commits messages, a documented branching model, and evidence of proper conflict resolution.
- Code quality is inconsistent across modules — the station module reflects best practice (service layer, centralized error handling), while the session module has a SQL-injection risk, missing validation, and inconsistent async patterns.
- Automated testing exists only on the backend and only partially covers edge cases; one existing test currently fails, correctly exposing a real defect.
- Documentation covers setup basics well but lacks an API reference, environment-variable explanations, and contributor guidelines.



7.3 Recommendations

Area	Recommendation	Priority
Code Quality	Refactor sessionController.js to use parameterized queries, a service layer, and centralized error handling consistent with stationController.js.	Critical
Code Quality	Adopt a linter/formatter (ESLint + Prettier) with a shared config and a pre-commit hook to enforce consistency automatically.	Major
Testing	Fix the failing “non-existent station” test by adding an existence check before using rows[0]; add frontend unit tests (Jest + React Testing Library).	Critical
Testing	Add integration tests that run against the docker-compose stack in CI to catch cross-service regressions.	Major

Area	Recommendation	Priority
Version Control	Encourage more frequent commits/pushes on long-running feature branches to reduce risk of lost work.	Minor
Documentation	Add an API reference section and a CONTRIBUTING.md as described in Section 6.2.	Major
Repository Management	Add a frontend tests/ folder and wire a coverage threshold into CI so both services are held to the same testing bar.	Major
Future Enhancement	Introduce static analysis (e.g., SonarQube or CodeQL) in CI to catch issues like the SQL-injection pattern automatically before merge.	Suggestion

The screenshot shows a web application for an EV charging system. The user is viewing their profile page. The sidebar on the left contains navigation options. The main form area contains several input fields for personal and vehicle information. The 'Profile' section is highlighted in the sidebar. The form includes fields for name, gender, address, city, state, vehicle number, brand, model, battery capacity, preferred charging type, and emergency contact. There are also buttons for saving the profile, resetting it, and returning to the dashboard.

7.4 Conclusion

Overall, the EV Charging Management System repository demonstrates strong foundational engineering discipline in its Git workflow and repository organization, which provide a solid base for collaborative development. The primary gap is consistency: the same standard of validation, error handling, and testing applied in the station module should be extended to the session and payment modules, and equivalent testing and documentation coverage should be brought to the frontend. Addressing the critical and major recommendations above would materially improve the codebase's reliability and long-term maintainability.

